



Self-Service Reporting & Analytics Platform

A cloud-native product that lets non-technical business users pull data from any source, merge and shape it with drag-and-drop, and publish pixel-perfect reports — without writing a single line of SQL.

INDUSTRY

ENGAGEMENT

DELIVERY MODEL

PLATFORM

**Higher-Education
Software**

~12-month program

**One-Team
(on/offshore)**

Serverless AWS

Confidentiality. Sanitized portfolio case study. The client and product names, proprietary business rules, and internal source are withheld under NDA. Screenshots are from a demo/test environment with client and product logos redacted. Figures describe delivery outcomes at a level safe for public sharing.

© 2026 Rohail K. Malhi — All rights reserved. Proprietary work; **not open-source**. The frameworks, architecture, metrics, and artifacts shown here may not be copied, adapted, or repurposed. Shared solely to evaluate the author's experience.

AT A GLANCE

CHALLENGE

Business users couldn't write SQL, so every report was a ticket to an overloaded engineering team. Backlogs grew; insight lagged days behind the question.

SOLUTION

A serverless, self-service platform: connect sources once, merge datasets visually, build no-SQL reports, and design pixel-perfect PDF/CSV output — backed by a Spark data lake.

RESULT

MVP + Post-MVP delivered and formally accepted over ~12 months; ~99% of tracked defects closed pre-release; reporting moved off the engineering critical path.

~12 mo

MVP + Post-MVP program, delivered & formally accepted

461/464

Internally-found defects resolved before release (~99%)

0

Lines of SQL required of the end user

3+

Source types unified: JDBC, CSV & reporting data blocks

CHALLENGE

Reporting was stuck behind engineering

The client — a software vendor serving large, data-heavy institutions — had customers sitting on data spread across operational databases, warehouses, and flat files. The people who actually needed reports (administrators, analysts, department heads) generally *couldn't* write SQL.

So every report request became a support ticket routed to a technical team. Turnaround was measured in days, the backlog never cleared, and the vendor's engineers spent their time hand-writing queries instead of building product. The client needed to put reporting **directly in the hands of business users** — safely, at scale, and without teaching everyone SQL.

- **No-SQL requirement:** end users must build real, joined, aggregated reports through a UI.
- **Many data sources:** relational databases, CSV/flat files, and third-party reporting data blocks — unified.
- **Scale & governance:** large datasets, multiple tenants/departments, and controlled access to shared reports.
- **Production-grade output:** not just tables — pixel-perfect, print-ready documents (e.g. transcripts and statements).

SOLUTION

A serverless, self-service reporting platform

We built a cloud-native product on AWS that removes engineering from the critical path. A user connects to their data once, then merges and shapes datasets and designs formatted reports entirely through a visual interface. What used to be a SQL-and-ETL task for engineers became a self-service action for the person who needs the answer.

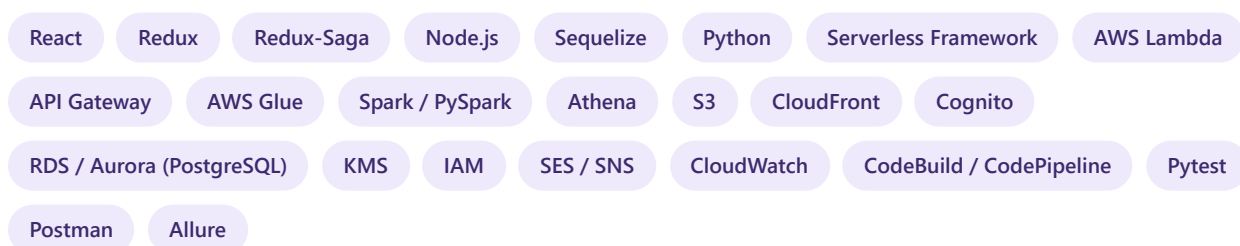
The delivery approach

- **Reusable Collections:** a dataset plus its metadata, so curated data is built once and reused across many reports — end users combine Collections, not raw tables.
- **Data Exchanger:** drag-and-drop join/merge of Collections with visual linking, plus filters, sorting, grouping, aggregate functions and calculated columns — with a live preview before commit.
- **Report & Template Builder:** turn a shaped dataset into a re-runnable report, then design a pixel-perfect layout (headers, footers, tables, repeating sections, print rules) with live data-field binding, exported to CSV and print-ready PDF.
- **Governance built in:** enterprise SSO/directory login, multi-tenant "Sites", Groups for shared read-only access, and per-request authorization.
- **Scale outside the request path:** heavy merges/aggregations run in a Glue + Spark data-lake pipeline over S3, queried via Athena — keeping the interactive app responsive.

Architecture at a glance

```
React SPA (edge-hosted: CloudFront + S3, Cognito auth)
|
| HTTPS
|
Serverless API (API Gateway + Lambda · Node.js/Sequelize + Python)
|
| authZ per request · collection / report / template services
|
├── RDS / Aurora
│   (PostgreSQL:
│   app metadata,
│   ACLs, definitions)
│   cross-cutting → KMS · IAM · SES/SNS · CloudWatch · VPC · CodePipeline
│
└── Data-Lake Pipeline
    (Glue Catalog/Jobs/Crawler · Spark/PySpark on S3
     · Athena query layer · external JDBC/CSV sources)
```

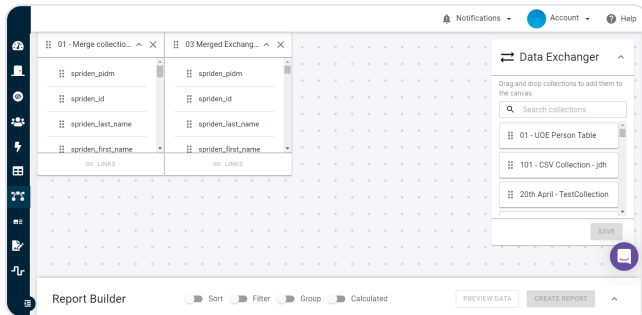
Technology



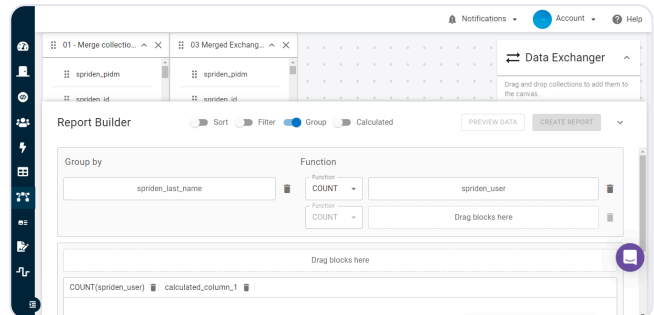
THE PRODUCT

Self-service, end to end

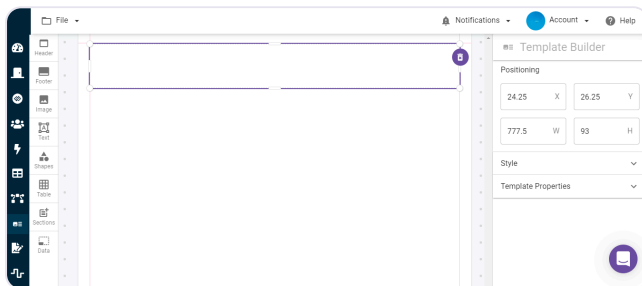
Demo/test environment. Client and product logos redacted for NDA.



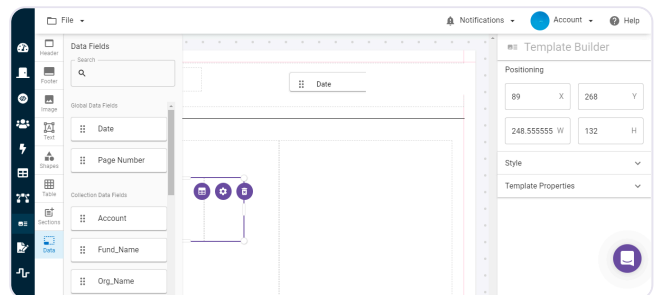
Data Exchanger — visually merge and join datasets on a canvas; no SQL.



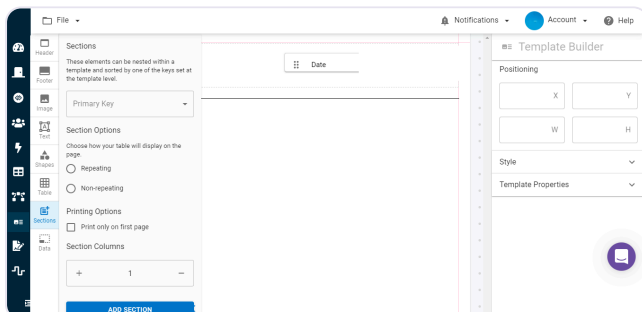
Report Builder — group-by, aggregate functions (COUNT/SUM) and calculated columns.



Template Builder — a pixel-perfect, drag-and-drop report designer.



Data binding — bind live data fields (and global fields) into the layout.



Sections & layout — repeating sections, multi-column, print rules.

RESULT**Delivered, accepted, and off the critical path****✓ Accepted**

Formal MVP acceptance from client stakeholders

~99%

461 of 464 internally-found defects closed before release

MVP+

Post-MVP added the pixel-perfect PDF template engine & encryption benchmarking

- **Reporting became self-service.** Non-technical users build joined, aggregated, formatted reports themselves — removing the engineering/SQL bottleneck from routine reporting.
- **Quality at delivery scale.** A Python/Pytest automation suite (API tests via Postman, Allure reporting) backed a multi-sprint delivery that closed hundreds of tracked defects across frontend, backend, and data-lake layers before release.
- **Production documents, not just tables.** The post-MVP Template Builder enabled pixel-perfect, data-bound PDF output (e.g. transcripts and statements) rendered server-side.
- **Serverless economics.** Fully serverless delivery (Serverless Framework + CodePipeline) gave usage-based scaling and no server fleet to maintain.

"We will accept the MVP delivery — I think you guys did a very, very good job, working through sketchy requirements. Integration of the [client] team into the project in the later stage worked really well."

— Product Manager, client (name withheld under NDA), on formal MVP acceptance

Note on figures. Metrics are drawn from the project's internal closure reporting and are presented in anonymized, aggregate form. Specific client data, dataset schemas, credentials, and identifying details are excluded under NDA.